

ES-FA / BTP Setup Notes

First local setup for the KV260-based demo

Updated May 5, 2026

Before starting

These notes are written as a practical setup record for the ES-FA / BTP project, not as a paper. The expected use case is simple: prepare the KV260 boot media, set up the local machine once, pull the **btp** repository, run the project, and generate the reports/plots/PDFs inside the local repo.

No project step here assumes cloud execution. Internet access is only needed to download official installers/images or to clone/pull the GitHub repository. All training outputs, Vivado/xsim logs, plots, runtime logs, and PDFs should stay under local repo folders such as **results/**, **paper_output/**, and **docs/**.

1 1. Prepare the KV260 SD Card First

1.1 Image to flash

Start with the board, because the rest of the setup is easier to understand once the target platform is clear. The KV260 has QSPI boot firmware on the SOM and uses the microSD card as the secondary boot device for Linux. For this project, flash an official Kria Ubuntu image onto the microSD card.

- First choice for a fresh setup: **Kria Ubuntu 24.04 LTS**.
- Use Ubuntu 22.04 only if a specific Kria app or tutorial you need still targets 22.04.
- For older boards, check boot firmware compatibility before blaming the SD card or project code. Newer Ubuntu images may require newer K26 boot firmware.

Download page:

<https://ubuntu.com/download/amd-xilinx>

1.2 Flashing steps

1. Download the Kria Ubuntu image for KV260/K26.
2. Install Raspberry Pi Imager or Win32 Disk Imager on the host machine.
3. Insert a 32 GB or larger microSD card.
4. Select the downloaded image and write it to the card.
5. Eject the card safely after the write completes.

1.3 First boot checklist

1. Insert the flashed microSD card into the KV260 carrier.
2. Connect Ethernet and either HDMI/keyboard or USB-UART serial.
3. Power the board.
4. Login with the default Ubuntu account if prompted:

```
username: ubuntu
password: ubuntu
```

5. Change the password when Ubuntu asks.
6. Update the board:

```
sudo apt update
sudo apt upgrade
```

7. Keep the serial console available during the first few boots. It is the fastest way to see SD-card or firmware boot problems.

2 2. Host Machine Requirements

The repo has been run from a Windows host, and the scripts also have Bash equivalents. For the presentation setup, the host machine should have:

- Python 3.10, 3.11, or 3.12;
- Git;
- AMD Vivado/Vitis if hardware validation is required;
- enough free disk space for **results/** and Vivado report folders.

Check the Xilinx tools locally:

```
vivado -version
xvlog -version
xelab -version
xsim -version
vitis -version
```

If these commands are not found, add the Xilinx **bin** folder to **PATH**, or set one of the environment variables that the helper scripts already search:

```
XILINX_VIVADO
XILINX_VITIS
XILINX_HOME
XILINX_INSTALL
VIVADO_HOME
VITIS_HOME
```

3 3. Get the btp Repository

The GitHub repository name is **btp**. Clone it like this:

```
git clone https://github.com/<your-github-user-or-org>/btp.git
cd btp
```

For an existing checkout:

```
cd btp
git status
git pull
```

If the folder was copied manually and is not a Git checkout, the code can still run. For long-term work, clone a clean **btp** repo and copy only the files you intentionally changed.

4 4. Python Setup

4.1 Windows PowerShell

```
cd btp
py -3.12 -m venv .venv312
.\.venv312\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The helper script does the pip part:

```
.\scripts\setup_env.ps1 -PythonExe .\.venv312\Scripts\python.exe
```

4.2 Linux or KV260 Ubuntu shell

```
cd btp
python3 -m venv .venv312
source .venv312/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Helper script:

```
PYTHON_EXE=python ./scripts/setup_env.sh
```

4.3 Quick sanity check

```
python -c "import torch, torchvision, matplotlib; import paper1_training.training_core;
print('imports ok')"
```

5 5. Run the Local Demo

5.1 Fast smoke run

For a presentation check, run the smoke pipeline first. This gives a quick end-to-end run without waiting for full Vivado implementation:

```
.\scripts\run_esfa_pipeline.ps1 -PythonExe .\venv312\Scripts\python.exe -Mode smoke -  
SkipVivado -SkipHardware
```

Bash equivalent:

```
MODE=smoke SKIP_VIVADO=1 SKIP_HARDWARE=1 ./scripts/run_esfa_pipeline.sh
```

5.2 Manual software run

Use this when you want to see the stages one by one:

```
python paper1_training/train_baseline.py  
python experiments/run_first2.py  
python experiments/run_remaining.py  
python iterations/run_all.py  
python analysis/compare.py  
python analysis/evidence_report.py
```

5.3 CLI demo

These are the small commands I use to show that the project has a system layer, not just training scripts:

```
python deployment/cli_interface/main.py --query "hi"  
python deployment/cli_interface/main.py --query "strike rate 167.55 balls 39"  
python deployment/cli_interface/main.py --query "classify results/runtime/sample_signal.  
bin"
```

6 6. Hardware Validation

After Vivado/xsim is available, run:

```
python hardware_validation/kv260/scripts/run_hw_validation.py --model-id baseline_paper1  
--scheduler-mode both --clock-mhz 100
```

If you only want simulator-side validation:

```
python hardware_validation/kv260/scripts/run_hw_validation.py --model-id baseline_paper1  
--scheduler-mode both --clock-mhz 100 --skip-vivado
```

The run folder is:

```
results/hardware_validation/<model-id>/<dense-or-event>/<run-id>/
```

The important files to check are `xsim_metrics.json`, `vivado_metrics.json`, `hardware_metrics.json`, and the logs/reports inside the same run folder.

7 7. Where Outputs Should Appear

Folder	What to look for
results/baseline/	baseline metrics, checkpoints, manifests, exports
results/experiments/	experiment metrics and exported artifacts
results/plots/	accuracy/energy/sparsity/latency/raster plots
results/analysis/evidence/	raw metric tables and percentage comparisons
results/hardware_validation/	xsim logs, Vivado reports, parsed hardware JSON
results/runtime/	CLI sample signal and adaptive decision logs
paper_output/	LaTeX report sources and compiled PDFs
docs/	setup notes, architecture notes, manual, checkpoints

8 8. Compile the PDFs Locally

Install a local LaTeX distribution such as MiKTeX or TeX Live. Then compile from the repository root:

```
pdflatex -interaction=nonstopmode -halt-on-error -output-directory=paper_output docs/  
first_time_user_setup_guide.tex  
pdflatex -interaction=nonstopmode -halt-on-error -output-directory=paper_output docs/  
first_time_user_setup_guide.tex  
pdflatex -interaction=nonstopmode -halt-on-error -output-directory=paper_output  
paper_output/progress_report.tex  
pdflatex -interaction=nonstopmode -halt-on-error -output-directory=paper_output  
paper_output/progress_report.tex
```

Expected PDFs:

- paper_output/first_time_user_setup_guide.pdf
- paper_output/progress_report.pdf

9 9. Common Problems

Problem	What I would check first
<code>git</code> is not recognized	Install Git and reopen the terminal.
<code>ModuleNotFoundError: torch</code>	Activate <code>.venv312</code> , then rerun <code>python -m pip install -r requirements.txt</code> .
PyTorch install fails	Use Python 3.10–3.12, not a preview Python build.
<code>vivado</code> or <code>xsim</code> not found	Check <code>PATH</code> or set <code>XILINX_VIVADO/XILINX_VITIS</code> .
<code>xsim</code> compile fails	Open <code>results/hardware_validation/.../logs/xvlog_*.log</code> .
Negative Vivado slack	This is a design/timing result, not a failed setup. Save it and use it for timing closure.
KV260 does not boot	Reflash the SD card, check serial output, and confirm boot firmware compatibility.
CLI cannot find signal file	Use a path relative to repo root, for example <code>results/runtime/sample_signal.bin</code> .
No plots are produced	Rerun <code>python analysis/compare.py</code> after checking that metric JSON files exist.

10 10. Presentation Checklist

1. KV260 SD card is flashed and the board boots Ubuntu.
2. Python environment is active and the import check passes.
3. Smoke pipeline completes locally.
4. `results/ranking_table.md` and `results/analysis_summary.json` exist.
5. Plots exist under `results/plots/`.
6. CLI examples return JSON output.
7. Hardware validation output exists if Vivado/xsim was available.
8. The setup guide and progress report PDFs are in `paper_output/`.

11 References Used for Board Setup

- AMD KV260 Ubuntu SD-card setup: https://xilinx.github.io/kria-apps-docs/kv260/2022.1/linux_boot/ubuntu_24_04/build/html/docs/sdcard.html
- Ubuntu AMD-Xilinx image download: <https://ubuntu.com/download/amd-xilinx>
- AMD Kria SOM and Starter Kit image/firmware matrix: <https://xilinx-wiki.atlassian.net/wiki/pages/viewpage.action?pageId=3307995140>
- AMD KV260 boot device overview: <https://docs.amd.com/r/en-US/ug1089-kv260-starter-kit/Boot-Devices-and-Firmware-Overview>